Midterm 01

ⓘ This is a preview of the published version of the quiz

Started: Oct 23 at 4:39pm

Quiz Instructions

This is Midterm 01. Once you start, you will have 1 hour and 30 minutes, but the test is designed to take less time. No partial credit is awarded, so be careful!

- Note that (unfortunately) the only fair way to proctor the exam is to do so without providing clarifications. If you feel that a question is ambiguous, please make a post to Campuswire after the exam and we will consider whether the grading of the question should be changed.
- If you have any technical issues during the exam (such as your internet going out) make a post to Campuswire. Don't worry: we'll figure something out.
- The exam is open book, open notes, open everything **except** you may not discuss the exam with another living organism until *after* the exam scores have been released.

Question 1 1 pts

Consider the problem of determining whether or not there is a pair of students in a class of n people who have the same birthday.

The brute force approach to this problem is to check each pair of people. What is the asymptotic time complexity of this approach?

- $\Theta(n^3)$
- $\Theta(n)$
- $\Theta(n^2)$

Question 2

1 pts

Identify a tight theoretical lower bound for the birthday problem described above. Your answer should be in terms of the class size, n.

- $\Omega(n)$
- $\Omega(n \log n)$
- $\Omega(n^2)$
- $\Omega(1)$

Question 3

1 pts

What is the asymptotic time complexity of the code below?

```
def create_empty_matrix(n):
    m = np.zeros((n, n))
    return m
```

- Theta(n)
- Theta(n^3)
- Theta(n^2)
- Theta(1)

Question 4

1 pts

What is the asymptotic time complexity of the code below?

```
def foo_1(n):
    for i in range(n):
        for j in range(n):
            for k in range(j):
                print('hi')
```

- Theta(n^3)
- Theta(n)
- Theta(n log n)
- Theta(n^2)

Question 5

1 pts

What is the asymptotic time complexity of the code below?

```
def factorial(n):
    r = 1
    i = n
    while i > 0:
        r = r * i
        i -= 1
    return r
```

- Theta(n)

- Theta(n^3)
- Theta(n!)
- Theta(n^2)

Question 6

1 pts

What is the asymptotic time complexity of the code below?

```
def middle(numbers):
    """n = len(numbers)"""
    count = len(numbers)
    return math.floor(count / 2)
```

- Theta(n^2)
- Theta(n^3)
- Theta(n)
- Theta(1)

Question 7

1 pts

What is the asymptotic time complexity of the code below?

```
def foo_2(n):
    i = 10 * n
    while i >= 1:
        i /= 2
        print('Hello world!')
```

- Theta(log n)
- Theta(2^n)
- Theta(1)
- Theta(n)
- Theta(n^2)

Question 8

1 pts

Suppose $f(n) = \Theta(n^3)$. True or False: there exist constants a, b, c, and d such that $f(n) = an^3 + bn^2 + cn + d$

- True
- False

Question 9

1 pts

Suppose the time complexity of a function "foo" is $O(n^3)$ and the time complexity of a function "bar" is $\Theta(n^4)$. What are the best bounds that can be placed on the time complexity of the following function?

```
def baz(n):
    foo(n)
    bar(n)
```

- Omega(n^3)
- O(n^4) and Omega(n^3)
- Omega(n^4)
- Theta(n^4)

Question 10

1 pts

This piece of code returns the number of "pairs" of the form (x, -x) in a collection of numbers:

```
def count_pairs(numbers):
    count = 0
    for x in numbers:
        if -x in numbers:
             count += 1/2
    return count
```

What is the time complexity of this code if the input is a Python list of size n?

- Theta(1)
- Theta(n log n)
- Theta(n^2)
- Theta(n)

Question 11

1 pts

True or False. If $f(n) = O(n^3)$ and $g(n) = \Omega(n^2)$, then $f(n)/g(n) = O(n)$.

- True
- False

Question 12

1 pts

What is the **best** case time complexity of the following code?

```
def foo_3(numbers):
    """n = len(numbers)"""
    for x in numbers:
        if x == 42:
            return sum(numbers)
```

- Theta(1)
- Theta(n)
- Theta(n^3)
- Theta(n^2)

Question 13

1 pts

True or False. Suppose an algorithm's best case time complexity is $T_{\text{best}}(n)$ and its worst case time complexity is $T_{\text{worst}}(n)$. Its *average* case time complexity is $O(T_{\text{worst}}(n))$ and $\Omega(T_{\text{best}}(n))$.

- True
- False

Question 14

1 pts

Given an n x d matrix representing n points in d dimensions, the *covariance* matrix is computed in numpy using the code below. What is its asymptotic time complexity?

```
mu = X.mean(axis=0)
1/(n-1) * (X - mu).T @ (X - mu)
```

- Theta(d n^2)
- Theta(n d^2)
- Theta(n d)
- Theta(n^2)

Question 15

1 pts

Suppose we have n data points in d-dimensional space and k "centers", also in d-dimensional space. What is the time complexity of the brute force approach to finding the center closest to each data point?

- Theta(n d^2)
- Theta(n k d)
- Theta(k^2 n d)
- Theta(n^2 k)

Question 16

1 pts

True or False: a quadratic algorithm may be faster than a linear algorithm on small enough inputs.

- True
- False

Question 17

1 pts

What is the time complexity of the following piece of code?

```
def recursive_search(numbers, target, start, stop):
    """Let n = stop - start"""
    if stop - start <= 0:
        return False
    middle = math.floor((start + stop) / 2)
    if numbers[middle] == target:
        return True
    in_left = recursive_search(numbers, target, start, middle)
    in_right = recursive_search(numbers, target, middle+1, stop)
    return in_left or in_right
```

- Theta(n log n)
- Theta(log n)
- Theta(n)
- Theta(n^2)

Question 18

1 pts

Here is a recursive algorithm for computing the factorial of n:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)'
```

What is the recurrence relation describing this function's run time?

- T(n) = T(n-1) + Theta(1)
- T(n) = n * T(n-1) + Theta(1)
- T(n) = n * T(n-1)
- T(n) = T(n/2) + Theta(n)

Question 19

1 pts

What is the **worst** case asymptotic time complexity of the following piece of code?

```
def search_for_both(arr, x, y):
    mergesort(arr)
    ix_x = binary_search(arr, x)
    ix_y = binary_search(arr, y)
    return (is_x is not None) and (ix_y is not None)
```

- Theta(log n)
- Theta(n log n)
- Theta(1)
- Theta(n)

Question 20

1 pts

What will happen when the following recursive function is run?

```
def minimum(arr):
    start = arr[0]
    if len(arr) == 1:
        return start
    rest = minimum(arr[0:])
    if start < rest:
        return start
    else:
        return rest
```

- An index error will occur.
- Infinite recursion.
- The minimum will be computed correctly.
- The maximum will be computed.

Quiz saved at 4:40pm Submit Quiz